Athel — 06 Build Loop, Review and Quality

Nov 1, 2026 · @Vaibhav Shukla

Each task is built by a developer agent and then checked by independent QA, security and review agents and by automated gates, looping until every gate passes or a limit sends it to a human.

Purpose, scope and user stories

This module turns an approved plan into code that a strict senior reviewer would accept. It covers features F-12 (build loop), F-13 (quality gates), F-14 (iterative review) and F-22 (parallel execution, later).

The quality bar is deliberately high: code that would pass review at a top engineering organisation, with every behaviour tested, every error path handled, and every decision traceable. Speed matters less than correctness.

User stories

ID	As a	I want	So that
US-BUILD-01	Developer	each task implemented in the repo's own style	the PR looks like our best engineer wrote it
US-BUILD-02	Reviewer	code reviewed by agents that did not write it	self-grading doesn't hide problems
US-BUILD-03	Reviewer	every review finding tracked until fixed or explicitly waived	nothing is quietly dropped
US-BUILD-04	Developer	tests written from the acceptance criteria, including failure paths	behaviour is proven, not assumed
US-BUILD-05	Security lead	every diff checked for vulnerabilities and risky dependencies	security is not an afterthought
US-BUILD-06	Developer	the loop to stop and ask me when it is stuck, not burn budget	I stay in control
US-BUILD-07	Tech lead	gates and thresholds set per repo in one file	quality is policy, not hope

Functional requirements

ID	Requirement	Acceptance criterion
FR-BUILD-001	Tasks run in dependency order from `plan.yaml`; a task starts only when its dependencies are `done`.	Scenario log shows topological order.
FR-BUILD-002	The developer agent receives only: the task, its spec requirements and acceptance criteria, `conventions.md`, relevant decisions and lessons, and the files in the task's paths.	Context manifest per task lists exactly these inputs.
FR-BUILD-003	Tests for each acceptance criterion are written before or with the implementation, follow the repo's existing test framework, folder layout, naming, fixtures and mocking style from conventions.md, and carry the AC ID in the name or a tag, whichever the repo's style allows.	Every AC ID appears in at least one test, and the reviewer confirms new tests match the existing test patterns.
FR-BUILD-004	QA adds negative, boundary and failure-path tests beyond the ACs (invalid input, timeouts, retries, concurrency where relevant).	QA report lists added cases per category.
FR-BUILD-005	Security review runs on every task's diff and on any dependency change.	Security report exists per task.
FR-BUILD-006	Review is done by a reviewer agent in a fresh context that sees the diff, spec and standards, but not the developer's reasoning.	Reviewer context manifest excludes developer transcript.
FR-BUILD-007	Every finding has an ID, severity, location, rule, and a required fix or a question; findings are closed only by a verified fix or a human waiver.	No task is `done` with open blocker or major findings.
FR-BUILD-008	After fixes, re-review checks the fixed findings and the new diff; it does not restart from zero.	Round N+1 report references round N findings.
FR-BUILD-009	All gates in `gates.yaml` for the repo must pass before a task is `done`.	`Stop` hook blocks completion while any gate is red.

ID	Requirement	Acceptance criterion
FR-BUILD-010	Fix-loop limits: 5 rounds per task, or 3 identical failures in a row, then a decision card.	Limit test raises a card, not an infinite loop.
FR-BUILD-011	Changes stay inside the task's paths; any need to edit elsewhere becomes a plan change request.	Out-of-path edit denied (doc 04 rule 7) and a change request created.
FR-BUILD-012	Commits are small and meaningful, one logical step each, with change and task trailers.	Commit lint passes.
FR-BUILD-013	Side findings noticed while building are recorded, never fixed inside the task (doc 07).	Fixture with a nearby bug yields a finding, no edit.
FR-BUILD-014	Gate waivers require a human, a reason and an expiry, and appear in the PR body.	Waiver without reason is rejected by the CLI.

Edge cases

Edge case	Required behaviour
Existing tests already failing on base	Record the baseline failures before the first edit; gates compare against baseline; never "fix" by deleting or skipping tests; raise a side finding
Flaky test	Re-run up to 3 times; if it passes inconsistently, mark flaky with evidence, side finding, and do not count it as the task's failure only if it also flakes on base
Repo has no tests or no test command	Onboarding detects it; plan includes a task to add a test harness first, or a card if that is out of scope
Gate tool not installed locally (linter, scanner)	Try the repo's own scripts and CI config; if unavailable, the gate is `unavailable`, which blocks unless a human waives it
Reviewer and developer disagree repeatedly on a finding	After 2 rounds of dispute, decision card with both arguments

Edge case	Required behaviour
Fix for one finding breaks another gate	The loop continues; the round counter still applies
Generated code (protobuf, OpenAPI clients)	Regenerate with the repo's generator, never hand-edit; review the generator input instead
Large refactor needed to implement the task cleanly	Stop; propose a separate refactor task through a plan change request
Coverage drops because of deleted dead code	Coverage gate uses changed-lines coverage, not only the total
Performance-sensitive path	If `risk_notes` names a latency budget, a benchmark or load test gate is added for that task
Dependency upgrade required	Separate task; security review of the new version; changelog checked for breaking changes

Role agents

Each role is a separate subagent with its own context, instructions and tool list; independence between the author and the checkers is the point.

Agent	Job	Inputs	Output (schema)	Tools
Developer	Implement one task and its AC tests; fix findings	Task, spec slice, conventions, decisions, lessons, task files, open findings	`build-result@1`: files changed, tests added, commands run, notes	Read, edit, write in worktree; build, test and lint commands; git commit
QA	Prove behaviour: add negative, boundary and failure tests; run the full suite	Task, ACs, diff, test files, conventions	`qa-report@1`: cases added by category, results, gaps	Read; write only under test paths; test commands

Agent	Job	Inputs	Output (schema)	Tools
Security	Find vulnerabilities in the diff and dependencies	Diff, dependency changes, threat checklist, data classification from spec	`security-report@1`: findings with CWE category and severity	Read; scanners (SAST, dependency audit, secret scan); no edit
Reviewer	Judge correctness, design, readability, conventions, spec fit	Diff, spec slice, conventions, lessons, previous rounds	`review@1`: findings, verdict	Read only
Architect	Called when a finding or change request needs a design decision	Spec, finding, investigation	Spec patch or decision card	Read; write only spec files through the CLI

Independence rules

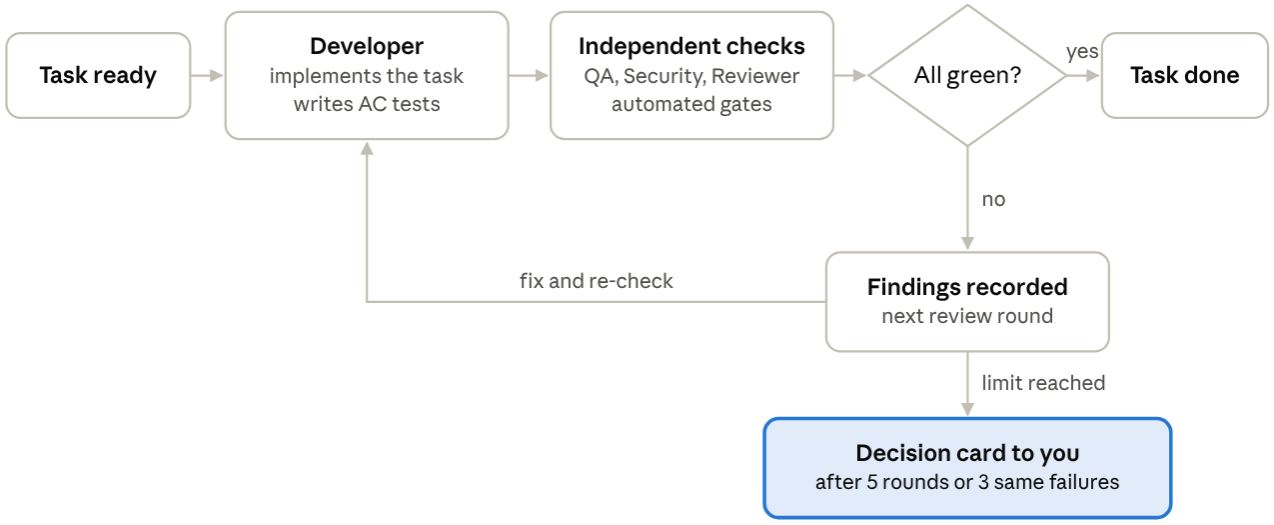
- The reviewer and security agents never see the developer's transcript or reasoning, only the result. This prevents them from adopting the developer's assumptions.
- A different system prompt and, where configured, a different model tier is used for review than for development.
- QA may write tests but never production code; if a test shows a bug, QA files a finding for the developer rather than fixing it.

Developer agent rules (excerpt from `agents/developer.md`)

1. Read `conventions.md`, the task, its ACs, decisions and lessons before editing anything.
2. Write or update tests for each AC in the repo's existing test style (framework, layout, naming, fixtures, mocks), then implement until they pass.
3. Handle every error path explicitly: no swallowed errors, no generic catch-all without logging and a typed error.
4. No TODOs, commented-out code, debug prints or disabled lint rules in the diff.
5. Keep functions small and names precise; follow existing patterns in the repo over personal preference.
6. Commit in small logical steps with trailers.
7. Stay inside the task's paths; ask for a plan change instead of editing elsewhere.
8. Never weaken a test or a gate to make it pass.

The per-task build loop

Every task loops through independent checks until all gates are green



build loop · one task, independent checks, fix rounds, limit to a human

A task enters the loop only when its dependencies are done and leaves it only with every gate green, or as a decision card when the round limit is reached.

Steps per round

1. `athel state enter build --task T-03` takes the task lock and records a checkpoint.
2. **Baseline (round 1 only):** run the repo's build and tests on the untouched worktree; store results so later failures can be told apart from pre-existing ones.
3. **Developer** implements and writes AC tests, commits in small steps.
4. **Checks** run in this order, each saving evidence: format and lint, build and types, unit tests, QA additions, security review and scans, contract diff, coverage, reviewer agent. Cheap checks first so expensive ones never run on broken code.
5. **Gate evaluation** by the helper CLI against `gates.yaml` (deterministic, no model judgement).
6. **All green:** task marked `done`, branch pushed, the draft PR updated. **Any red:** findings written for the next round, developer fixes, the loop repeats.
7. The `Stop` hook refuses to end the step while the task is neither done nor blocked.

Iterative review protocol

Review runs in rounds; each round reviews the new diff plus the fixes for the previous round's findings, and a task can only finish when no blocker or major finding is open.

Severity

Severity	Meaning	Must be closed before done
Blocker	Wrong behaviour, spec violation, security issue, data loss risk, broken contract	Yes, fixed
Major	Missing error handling, missing tests for a path, convention breach that harms maintainability, performance risk	Yes, fixed or waived by a human
Minor	Naming, small readability issues, local simplifications	Fixed if cheap; otherwise listed in the PR
Nit	Style preference within conventions	Optional; never blocks

Finding format (`reviews/<task>/round-N.yaml`)

```
- id: R2-04
  severity: major
  source: reviewer            # reviewer | security | qa | gate
  location: src/profile/phone.ts:42-58
  rule: errors.no-silent-catch
  finding: Parser exception is caught and returns null; caller treats null as
"no change".
  required: Throw PhoneInvalidError and map to 400 PHONE_INVALID per AC-2.
  status: open                # open | fixed | verified | waived | disputed
  history: [{round: 2, status: open}]
```

Round rules

1. Round 1 is a full review. Later rounds review the fixes and the new diff only, plus a check that fixes didn't break earlier-verified areas.
2. The developer marks a finding `fixed` with the commit; only the reviewer moves it to `verified`.
3. A `disputed` finding needs a written reason from the developer; two rounds of dispute raise a decision card.
4. A round with zero new blocker or major findings and all previous ones verified ends review for the task.
5. Limit: 5 rounds per task (FR-BUILD-010).

Review checklist (reviewer agent, applied every round)

- Spec fit: each changed behaviour maps to a requirement; nothing extra.
- Correctness: edge cases, null and empty, concurrency, idempotency, time zones, units.

- Errors: typed errors, no silent catch, meaningful messages, correct status codes.
- Contracts: backward compatibility as planned; versioning respected.
- Data: migrations reversible or flagged; no N+1 queries; indexes for new queries.
- Observability: logs at the right level without PII, metrics for new paths, trace context passed on.
- Security: input validation, authz checks, no secrets, safe defaults.
- Tests: AC coverage, failure paths, deterministic, no sleeps, no reliance on order, same framework and style as the repo's existing tests.
- Readability: names, function size, comments explain why not what, conventions followed.

Quality gates and the code quality standard

Gates are commands with pass criteria, defined per repo in `gates.yaml`; they run in the task's worktree, and their results are saved as evidence that goes into the PR.

Default gates (`gates.yaml`)

Gate	Pass criterion (default)	Waivable by
build	Compiles or bundles with zero errors	Never
unit	All tests pass; no tests skipped or deleted versus baseline	Never
integration	Repo's integration suite passes where one exists	Human, with reason
lint	Zero new errors; no new disable comments	Human
types	Zero new type errors (typed languages)	Never
format	Formatter reports no changes	Never (auto-fixed)
coverage	Changed-lines coverage ⩾ 90%; total does not drop by more than 0.5 points	Human
mutation (HIGH risk tasks)	Mutation score on changed code ⩾ 70%	Human
contract	API and event schema diffs match the plan (additive unless planned otherwise)	Never

Gate	Pass criterion (default)	Waivable by
security-scan	No new high or critical SAST findings; no new vulnerable dependencies; no secrets	Security owner only
complexity	No new function with cyclomatic complexity over 15 or longer than 80 lines	Human
migration	Migration runs up and down on a disposable database; online-safe checks pass	Never
performance (when a budget is set)	Benchmark within budget from `risk_notes`	Human
review	No open blocker or major findings	Never

Code quality standard (what reviewers and gates enforce together)

1. **Correctness first:** every requirement has a test; every error path is handled and tested.
2. **Explicit failure:** typed errors, no silent catches, no magic return values meaning failure.
3. **Defensive at boundaries, simple inside:** validate all external input at the edge; trust validated data inside.
4. **Idempotent and retry-safe** handlers for anything called over the network or by a queue.
5. **Observability:** structured logs without PII, metrics for new endpoints and queues, trace propagation.
6. **Backward compatibility** by default for every contract and schema.
7. **No dead weight:** no TODOs, commented code, unused code, debug output, or new dependencies without a reason.
8. **Readable:** small functions, precise names, comments for why; the repo's conventions win over personal style.
9. **Deterministic tests:** no sleeps, no real network, no shared state between tests, fixed seeds and clocks.
10. **Secure defaults:** least privilege, parameterised queries, output encoding, safe deserialisation.

Each rule has an ID (`errors.no-silent-catch`, etc.) used in review findings, so patterns can be counted and turned into lessons (doc 07).